



การสร้างและใช้งาน COMPONENT

030513702 : Selected Topics in Software Development
Asst. Prof. Dr. Phollakrit Wongsantisuk

ทบทวนพื้นฐานเกี่ยวกับคลาส

ตั้งแต่ ECMAScript 6 เป็นต้นมา ได้รองรับการสร้างคลาส (Class) รวมถึงลักษณะทางด้าน OOP ที่ใกล้เคียงกับภาษาอื่นๆ โดยมีรูปแบบพื้นฐานคือ

```
class ชื่อคลาส {  
    //สมาชิกภายในคลาส  
}
```

เรานิยมให้ชื่อคลาสขึ้นต้นด้วยตัวพิมพ์ใหญ่ ส่วนหลักการอื่นๆ ก็คล้ายกับการตั้งชื่อตัวแปร เช่น

```
class Product {  
    ...  
}
```

เมื่อเรานำคลาสมาใช้งาน จะต้องสร้างตัวแปรที่ใช้อ้างอิงกับคลาสซึ่งเราเรียกว่า อินสแตนซ์ (Instance) พร้อมกับใช้คำสั่ง new แล้วตามด้วยชื่อคลาสและวงเล็บ () เพื่อสร้างอินสแตนซ์ เช่น

```
const p = new Product()    //การใช้ const จะเหมาะสมกว่า let และ var
```

ทบทวนพื้นฐานเกี่ยวกับคลาส

- **พรีอ็อปเพอร์ตีและคอนสตรัคเตอร์**

พรีอ็อปเพอร์ตี (Property) ก็คือข้อมูลประจำตัวของออบเจกต์ ถ้าข้อมูลดังกล่าวมีค่าที่แน่นอน เราอาจกำหนดไว้ล่วงหน้าโดยไม่ต้องระบุ `const` หรือ `let` หรือ `var` ในลักษณะดังนี้

```
class Circle {  
    PI = 3.141  
}
```

แต่หากข้อมูลดังกล่าว ต้องเปลี่ยนแปลงตามการสร้างอินสแตนซ์ในแต่ละครั้ง เราต้องรับข้อมูลดังกล่าวเข้ามาทางเมธอดพิเศษที่ชื่อ **constructor** พร้อมระบุพารามิเตอร์ที่จะใช้ในการรับข้อมูล แล้วจัดเก็บให้เป็นพรีอ็อปเพอร์ตีของคลาสดังรูปแบบต่อไปนี้

```
class Test {  
    constructor(parameterName) {  
        this.<propertyName> = parameterName  
    }  
}
```

ทบทวนพื้นฐานเกี่ยวกับคลาส

ชื่อพรีอปเพอร์เตอร์อาจเป็นชื่อเดียวกันกับชื่อพารามิเตอร์ก็ได้ เช่น กรณีของคลาส Circle เราอาจรับค่ารัศมีเข้ามาทางคอนสตรัคเตอร์แล้วจัดเก็บไว้ในแบบพรีอปเพอร์เตอร์ชื่อ radius ดังนี้

```
class Circle {  
    constructor(radius) {  
        this.radius = radius  
    }  
}
```

ในแต่ละคลาส จะมีคอนสตรัคเตอร์ได้เพียงอันเดียว ดังนั้น เราอาจรับข้อมูลที่จำเป็นเข้ามามากกว่า 1 ค่า โดยใช้หลักการเดียวกับพารามิเตอร์ของฟังก์ชัน เช่น

```
class Person {  
    constructor(name, career, yearBorn, nationality='Thai') {  
        this.name = name  
        this.career = career  
        this.age = new Date().getFullYear() - yearBorn  
        this.nationality = nationality  
    }  
}
```

ทบทวนพื้นฐานเกี่ยวกับคลาส

เมื่อสร้างอินสแตนซ์ของคลาส ต้องระบุพารามิเตอร์ให้ครบทั้งหมด ยกเว้นตัวที่มีค่าดีฟอลต์ เช่นเดียวกับการเรียกฟังก์ชัน จากนั้นก็สามารถอ่านหรือแก้ไขค่าของพร็อพเพอร์ตี้ ได้ตามต้องการ เช่น

```
const circle = new Circle(10)
const person = new Person('Somsri', 'Accountant', 1990)
```

ทบทวนพื้นฐานเกี่ยวกับคลาส

- **เมธอดของคลาส**

เมธอด (Method) คือ วิธีการกระทำอย่างใดอย่างหนึ่งของคลาส แม้ส่วนใหญ่จะใช้หลักการเดียวกับฟังก์ชัน แต่ก็มีบางอย่างที่แตกต่างออกไป ซึ่งสามารถสรุปได้ดังนี้

- ให้ระบุเฉพาะชื่อเมธอด โดยไม่ต้องมีคำว่า **function** นำหน้า
- การกำหนดพารามิเตอร์ ใช้หลักการเดียวกับของฟังก์ชัน
- ภายในเมธอด หากต้องการอ้างถึงพร็อพเพอร์ตี้หรือเมธอดอื่นๆ ภายในคลาสเดียวกัน ก็ใช้รูปแบบ `this.<propertyName>` หรือ `this.<methodName>`

```
class ThaiDate {  
    constructor(year, month, date) {  
        this.year = year  
        this.month = month  
        this.date = date  
    }  
    getShortDate() { //เมธอดชื่อ getShortDate  
        return `${this.date}/${this.month}/${this.year+543}`  
    }  
    getLongDate() { //เมธอดชื่อ getLongDate  
        const m = ['มกราคม', 'กุมภาพันธ์', ..., 'ธันวาคม']  
        return `${this.date} ${m[this.month-1]} ${this.year+543}`  
    }  
}
```

ทบทวนพื้นฐานเกี่ยวกับคลาส

- การสืบทอดระหว่างคลาส

เราสามารถสืบทอดจากคลาสหนึ่งมาสู่อีกคลาสหนึ่งได้ โดยใช้รูปแบบดังต่อไปนี้

```
class Subclass extends Superclass {  
    ...  
}
```

- Subclass คือ คลาสผู้สืบทอด หรือคลาสใหม่ที่เรา กำลังจะสร้างขึ้น
- Superclass คือ คลาสต้นแบบ หรือคลาสเดิมที่มีอยู่แล้ว
- extends เป็นคีย์เวิร์ดที่บ่งชี้ว่า Subclass จะสืบทอดมาจาก Superclass

ทบทวนพื้นฐานเกี่ยวกับคลาส

หลังการสืบทอด Subclass จะมีองค์ประกอบเช่นเดียวกับ Superclass และเราสามารถเพิ่มเมธอดรวมถึงพรีอเพอร์เตอร์ให้กับ Subclass ได้ เช่น

```
class SimpleDate {  
    constructor(year, month, date) {  
        this.year = year  
        this.month = month  
        this.date = date  
    }  
    getShortDate() { //เมธอดชื่อ getShortDate  
        return `${this.date}/${this.month}/${this.year+543}`  
    }  
}
```

ทบทวนพื้นฐานเกี่ยวกับคลาส

```
class ThaiDate extends SimpleDate {  
  getLongDate() {      //เมธอดที่เพิ่มใหม่  
    const m = ['มกราคม', 'กุมภาพันธ์', ... , 'ธันวาคม']  
    return `${this.date} ${m[this.month-1]} ${this.year+543}`  
  }  
  isLeapYear() {      //เมธอดที่เพิ่มใหม่  
    let y = this.year  
    if ((y%400==0) || ((y%4==0) && (y%100!=0))) {  
      return true  
    }  
    return false  
  }  
}
```

```
const thDate = new ThaiDate(2022, 12, 31)  
console.log(thDate.getShortDate())    //เรียกใช้เมธอดที่อยู่ใน Superclass ได้  
console.log(thDate.getLongDate())     //เรียกใช้เมธอดที่สร้างเพิ่มใน Subclass
```

ทบทวนพื้นฐานเกี่ยวกับคลาส

- การโอเวอร์ไรด์เมธอดของ Superclass

นอกจากการเพิ่มเมธอดใหม่แล้ว เราอาจกำหนดวิธีการทำงานของเมธอดเดิมที่มีใน Superclass ให้เป็นแบบใหม่ตามต้องการ ซึ่งเราเรียกกรณีนี้ว่า การโอเวอร์ไรด์ (Override) โดยข้อกำหนดที่สำคัญ คือ

- ต้องเขียนชื่อเมธอดและรูปแบบตัวพิมพ์ให้ตรงกับใน Superclass
- รูปแบบการรับพารามิเตอร์ต้องตรงกัน

หากไม่ครบตามข้อกำหนดของ 2 ข้อดังที่กล่าวมา ถือเป็นการสร้างเมธอดใหม่ เพิ่มเข้ามาใน Subclass เช่น เมธอด `getShortDate()` นั้นมีอยู่แล้วในคลาส `SimpleDate` แต่เราต้องการเปลี่ยนแปลงใหม่ จึงโอเวอร์ไรด์ในคลาส `ThaiDate` เป็นดังนี้

```
class ThaiDate extends SimpleDate {  
    //...  
    getShortDate() {        //Override  
        return `${this.date}-${this.month}-${this.year+543}`  
    }  
}
```

ทบทวนพื้นฐานเกี่ยวกับคลาส

- **การสร้างคอนสตรักเตอร์ให้แก่ Subclass**

ปกติแล้ว คอนสตรักเตอร์ของ Superclass ก็จะถูกสืบทอดมายัง Subclass เช่นเดียวกับเมธอด ดังนั้นการสร้างอินสแตนซ์ของ Subclass ก็ต้องระบุพารามิเตอร์ให้ครบตามที่กำหนดไว้ในคอนสตรักเตอร์ของ Superclass แต่อย่างไรก็ตาม หากใน Subclass เราต้องการใช้ข้อมูลบางอย่างที่แตกต่างไปจาก Superclass ก็อาจสร้างคอนสตรักเตอร์ขึ้นมาใหม่ได้ โดยมีข้อกำหนดที่สำคัญดังนี้

- ที่คอนสตรักเตอร์ของ Subclass ต้องส่งข้อมูลให้แก่คอนสตรักเตอร์ของ Superclass โดยใช้คีย์เวิร์ด `super` พร้อมระบุข้อมูลให้ครบตามที่ได้กำหนดไว้ในคอนสตรักเตอร์ของ Superclass เช่น `super(1, true, 'three')`
- การส่งข้อมูลให้แก่ Superclass ด้วยคีย์เวิร์ด `super` ต้องดำเนินการเป็นลำดับแรก นั่นคือ ในคอนสตรักเตอร์ของ Subclass เราต้องกำหนดพารามิเตอร์ด้วย `super()` เป็นคำสั่งแรกเสมอ

เช่น สมมติว่าที่คลาส `ThaiDate` จากหัวข้อที่แล้ว หากเราจะกำหนดคอนสตรักเตอร์ให้แก่ Subclass โดยนอกเหนือจากวันเดือนปีแล้ว ก็ให้สามารถระบุชั่วโมงและนาทีได้ด้วย ก็อาจสร้างคอนสตรักเตอร์ดังนี้

ทบทวนพื้นฐานเกี่ยวกับคลาส

```
class ThaiDate extends SimpleDate {  
    constructor(year, month, date, hour=0, minute=0) {  
        super(year, month, date) //ส่งค่าให้แก่คอนสตรักเตอร์ของ SimpleDate  
        this.year = year  
        this.month = month  
        this.date = date  
        this.hour = hour  
        this.minute = minute  
    }  
    //...  
}
```

Arrow Function

Arrow Function เป็นการสร้างฟังก์ชันอีกแบบหนึ่ง ที่เขียนให้สั้นและกระชับขึ้นกว่ารูปแบบเดิม ซึ่งรูปแบบที่เทียบเท่าหรือมีวัตถุประสงค์คล้ายคลึงกันในภาษาคอมพิวเตอร์อื่นๆ เช่น ใน Java, C#, Python จะเรียกว่า Lambda Expression แต่ในจาวาสคริปต์จะเรียกว่า Arrow Function ตามสัญลักษณ์ที่ใช้ ดังรูปแบบพื้นฐานต่อไปนี้

```
(พารามิเตอร์) => {  
    //คำสั่งต่างๆ  
}
```

- พารามิเตอร์ ก็เหมือนกับที่เรากำหนดให้แก่ฟังก์ชันทั่วไป ซึ่งอาจเป็นพารามิเตอร์ธรรมดา หรือ Default Parameter หรือ Rest Parameter ก็ได้
- => เป็นสัญลักษณ์ที่จาวาสคริปต์ใช้ในการกำหนด Arrow Function
- คำสั่งต่างๆ หรือเรียกว่าส่วน Body สำหรับกำหนดการทำงานตามที่เราต้องการ

Arrow Function

ในการใช้งาน เราอาจต้องสร้างตัวแปรหรือค่าคงที่ มาใช้อ้างอิงกับฟังก์ชันเช่นเดียวกับกรณีของ Anonymous Function (ฟังก์ชันที่ไม่มีการตั้งชื่อ) รวมถึงวิธีการเรียกใช้งานก็ทำแบบเดียวกัน ดังแนวทางต่อไปนี้

```
var square = (n) => {  
    if (Number.isFinite(n)) {  
        return n ** 2  
    }  
}
```

```
var repeatString = (str, times, separator=' ') => {  
    if (!Number.isInteger(times)) {  
        return  
    }  
    for (i=0; i<times; i++) {  
        console.log(str + separator)  
    }  
}  
  
var a = square(10)  
console.log(square(5))  
repeatString('*', 10)
```

Arrow Function

จากโค้ดจะพบว่า การเขียน Arrow Function ก็เหมือนกับการตัดคำว่า function และชื่อของมันออกไปจากรูปแบบปกติ แล้วนำสัญลักษณ์ => มาเชื่อมระหว่างพารามิเตอร์และบล็อก { } ในรูปแบบ (...) => { ... } แค่นั้นเอง อย่างไรก็ตาม บางกรณีเราสามารถเขียน Arrow Function แบบลัดรูป ให้กระชับขึ้นกว่าเดิม ดังแนวทางต่อไปนี้

- ถ้ามีพารามิเตอร์เพียงตัวเดียว เราไม่จำเป็นต้องเขียนวงเล็บครอบพารามิเตอร์ก็ได้ เช่น

```
var increment = n => { return ++n }  
//แต่กรณีที่ไม่มีพารามิเตอร์เลย ต้องเขียนเป็นวงเล็บว่าง เช่น  
var newline = () => { console.log('\n') }
```

- ในส่วน Body ถ้ามีเพียงคำสั่งเดียว และไม่ส่งค่ากลับด้วยคีย์เวิร์ด return เราอาจไม่เขียนบล็อก { ... } ครอบคำสั่งนั้นก็ได้ เช่น

```
var print = str => console.log(str)
```

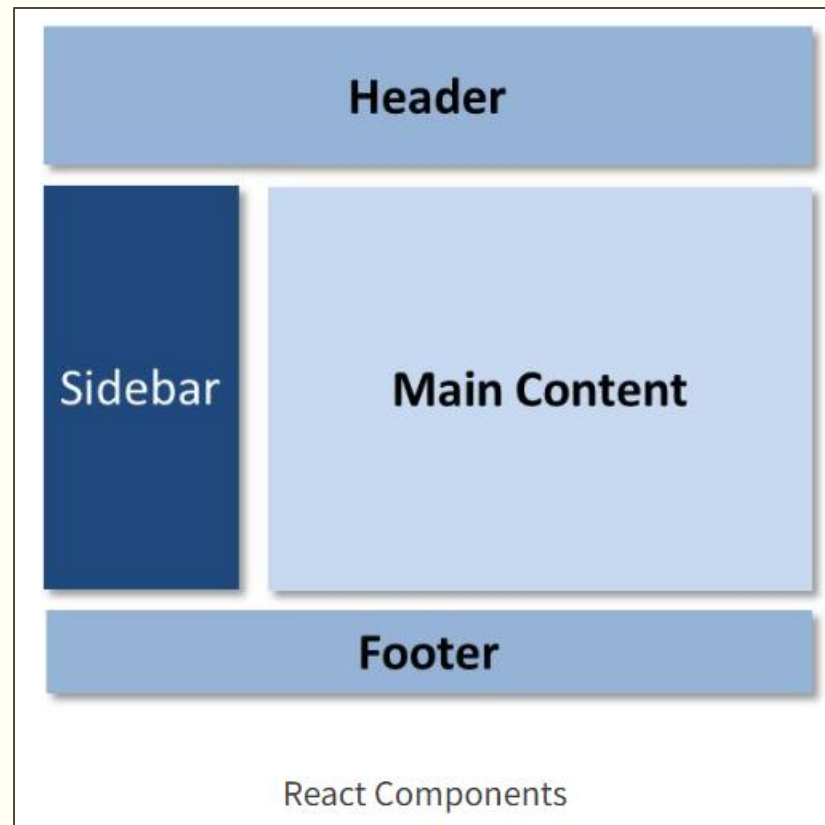
Arrow Function

- ถ้าในส่วน **Body** มีคำสั่งเดียว จาวาสคริปต์จะมองว่า เราจะส่งผลลัพธ์หรือค่าที่เกิดขึ้นกลับออกไปเสมอ ดังนั้นจึงอาจจะคีย์เวิร์ด `return` ก็ได้ แต่ถึงแม้เราจะไม่ส่งค่าใดๆ กลับไป การไม่ระบุค่าคีย์เวิร์ด `return` ก็ไม่ก่อให้เกิดข้อผิดพลาด (แต่ต้องเป็นกรณีที่มีเพียงคำสั่งเดียวเท่านั้น)

```
var printLine = str => console.log(str)
var decrement = n => --n //ส่งค่า --n กลับออกไปโดยอัตโนมัติ
```

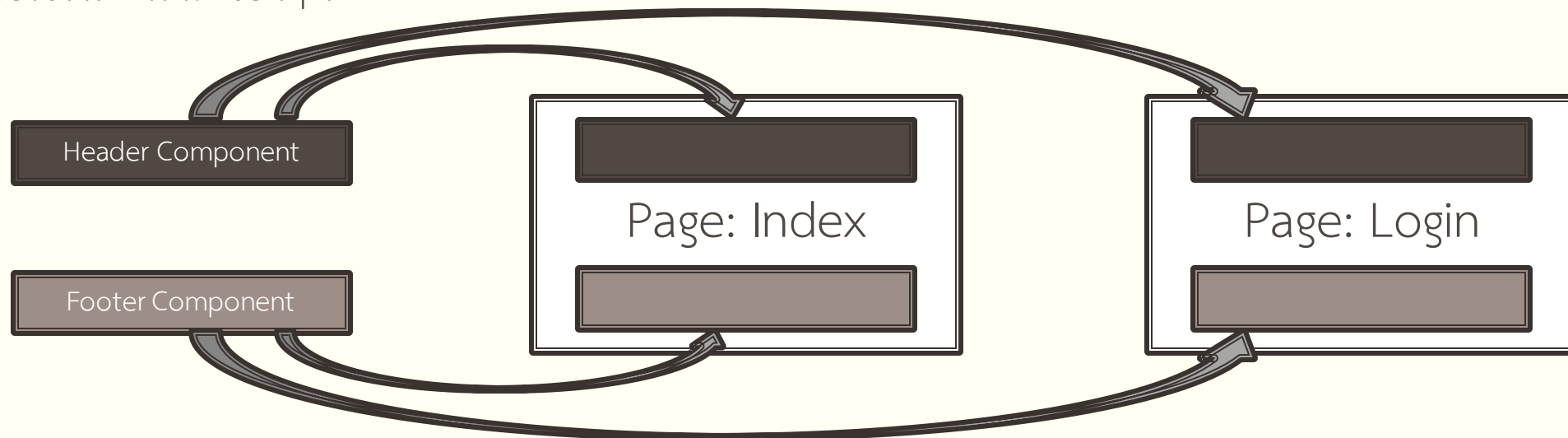
หลักการของ React Component

การสร้าง Web Application ตามแนวทางของ React ก็คือ จะแบ่งองค์ประกอบภายในเว็บเพจออกเป็น ส่วนย่อยๆ ที่เรียกว่า คอมโพเนนต์ (Component) เช่น ส่วน Navigation (Menu), ส่วนของ Content ปลีกย่อยสำหรับ เนื้อหาในแต่ละเรื่อง และส่วนของ Footer เป็นต้น ซึ่งเมื่อนำส่วนต่างๆ เหล่านี้มารวมกัน ก็จะกลายเป็นเว็บเพจที่สมบูรณ์ โดยให้กรอบสี่เหลี่ยมแต่ละอันคือคอมโพเนนต์ที่นำมารวมกันเป็นเว็บเพจ



หลักการของ React Component

ตามปกติแล้ว เราจะสร้างคอมโพเนนต์ไว้ในไฟล์แยกต่างหาก เพื่อจะนำไปใช้ร่วมกันในเพจอื่นๆ เช่น สมมติว่าเราสร้างคอมโพเนนต์ Header สำหรับแสดงส่วนหัวของเพจ และคอมโพเนนต์ Footer สำหรับส่วนล่างของเพจ ก็สามารถนำไปใช้ร่วมกันในเพจอื่นๆ ได้



การสร้างคอมโพเนนต์ใน React สามารถทำได้ 2 แบบคือ

- **Function Component** เป็นการสร้างคอมโพเนนต์ในแบบฟังก์ชัน
- **Class Component** เป็นการสร้างคอมโพเนนต์ในแบบคลาส

การสร้างคอมโพเนนต์ในแต่ละแบบ จะมีข้อกำหนดปลีกย่อยที่แตกต่างกันในบางกรณี ดังรายละเอียดที่จะกล่าวถึง

ต่อไป

การสร้าง Function Component

หลักการโดยส่วนใหญ่ของ Function Component ก็เหมือนกับการสร้างฟังก์ชันแบบปกติของ JavaScript แต่ลักษณะปลีกย่อยบางอย่างที่เราควรรู้เพิ่มเติมรวมถึงแนวทางการนำไปใช้งานที่แตกต่างจากฟังก์ชันปกติ ดังนี้

- ไฟล์ที่เราจะสร้าง Function Component อย่างน้อยต้องนำเข้าคอมโพเนนต์ React จากโมดูล 'react' เช่น `import React from 'react'`
- เพื่อให้แตกต่างจากฟังก์ชันแบบปกติทั่วไป ส่วนใหญ่เรานิยมตั้งชื่อ Function Component ในแบบ PascalCase หรือให้ทุกคำขึ้นต้นด้วยตัวพิมพ์ใหญ่นั่นเอง
- เราสามารถสร้าง Function Component ในแบบฟังก์ชันปกติ (Regular Function) หรือจะสร้างแบบ Arrow Function ก็ได้
- Function Component ต้องส่งค่ากลับคืนในแบบ JSX ด้วยคำสั่ง `return` เพื่อนำไปรวมกับคอมโพเนนต์ตัวอื่นๆ ให้กลายเป็นเว็บเพจ
- โดยทั่วไป เรามักจะสร้าง Function Component แยกไว้ในไฟล์ต่างหาก แล้วนำไปใช้ที่ไฟล์อื่นๆ ในแบบของโมดูล ดังนั้น ภายในไฟล์ที่สร้าง Function Component ต้องส่งออก (export) ฟังก์ชันด้วย

เราลองสร้างคอมโพเนนต์ Header สำหรับแสดงส่วนหัวของเพจ แล้วก็คอมโพเนนต์ Content สำหรับแสดงเนื้อหาของเพจ และคอมโพเนนต์ Footer สำหรับแสดงส่วนล่างของเพจ โดยสร้างไฟล์ใหม่เข้ามาในโฟลเดอร์ `src` ซึ่งจะตั้งชื่อไฟล์เป็น `func-components.js`

การสร้าง Function Component

ตัวอย่างการใช้งาน

react/app1/src/func-components.js

```
import React from 'react'

export function Header() {
  const aStyle = {
    display: 'inline-block',
    /* กำหนดให้สีตัวอักษรเป็นสีฟ้า */
    margin: '7px',
    /* กำหนดให้ลิงก์ไม่ต้องมีเส้นขีดใต้ */
  }
```

```
  return (
    <div style={{backgroundColor: '#ccc'}}>
      <a href=" " style={.....}>.....</a> |&nbsp;
      <a href=" " style={.....}>.....</a> |&nbsp;
      <a href=" " style={.....}>.....</a>
    </div>
  )
}
```

การสร้าง Function Component

```
export function ..... {  
  return (  
    <div style={{textAlign: .....}}>  
      <h2>.....</h2>  
      <br/><br/><br/>  
    </div>  
  )  
}  
  
export const Footer = () => {  
  return (  
    <div style={{textAlign: .....}}>  
      &copy; {new Date().getFullYear()} .....  
    </div>  
  )  
}
```

การสร้าง Function Component

เพจหรือไฟล์ที่ต้องการนำ Function Component ไปแสดงผล อันดับแรก ต้องนำเข้าคอมโพเนนต์ในรูปแบบโมดูลจากไฟล์ที่จัดเก็บมันเอาไว้

ตัวอย่างการใช้งาน

react/app1/src/App.js

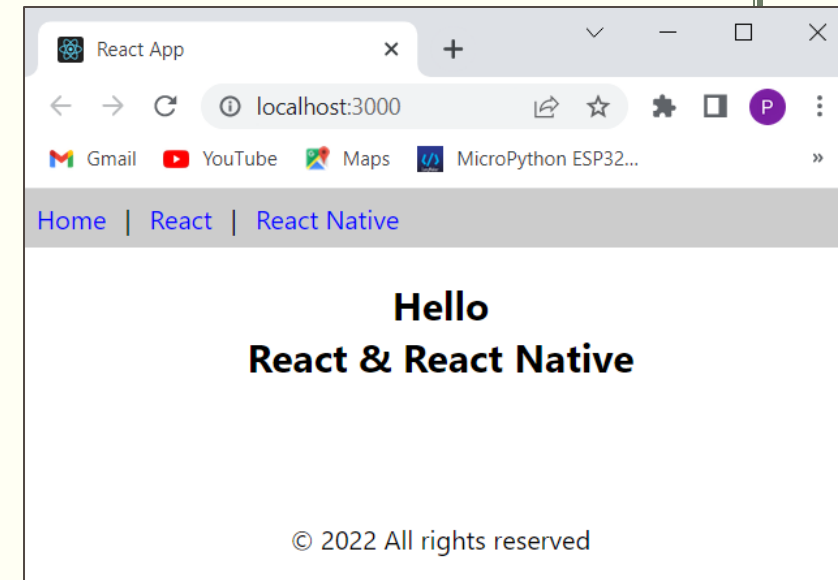
```
import React from 'react'
import { ..... } from './func-components'

function App() {
  //แบบที่ 1 กำหนดฟังก์ชันทั้งหมดที่จะแสดงในแบบอาร์เรย์
  return (
    [....., <Content />, .....]
  )
}
```

//แบบที่ 2 กำหนดฟังก์ชันทั้งหมดที่จะแสดงลงใน Parent เดียวกัน

```
/*
return (
  <>
    <Header/>
    <Content/>
    <Footer/>
  </>
)
*/
}

export default App
```



การสร้าง Class Component

Class Component ก็คือคอมโพเนนท์ที่สร้างในรูปแบบของคลาส แต่อย่างไรก็ตามเพื่อให้คลาสดังกล่าวตรงตามหลักการของ React จึงมีข้อกำหนดบางส่วนเพิ่มเติมเข้ามา นอกเหนือจากวิธีการตามปกติของคลาสในแบบของ JavaScript ซึ่งสามารถสรุปประเด็นที่สำคัญได้ดังนี้

- ไฟล์ที่จะสร้าง Class Component อย่างน้อยต้องนำเข้าคอมโพเนนท์ React จากโมดูล 'react' เช่น import React from 'react'
- คอมโพเนนท์ที่จะสร้างขึ้น ต้องสืบทอดจากคลาสที่ชื่อ Component เสมอ ซึ่งเราอาจใช้วิธีใดวิธีหนึ่งดังนี้

//วิธีที่ 1

```
import React from 'react'  
Class MyComp extends React.Component { //อ้างถึงคลาส Component ผ่าน React  
  ...  
}
```

//วิธีที่ 2 นำเข้าคลาส Component โดยตรง

```
import React, {Component} from 'react'  
Class MyComp extends Component { //อ้างถึงคลาส Component โดยตรง  
  ...  
}
```

การสร้าง Class Component

- ภายในคลาส มีข้อบังคับที่สำคัญ คือ ต้องโอเวอร์ไรด์เมธอด render() แล้วภายในเมธอดดังกล่าวก็กำหนดสิ่งที่แสดงผลในรูปแบบโค้ด JSX แล้วส่งออกไปด้วยคำสั่ง return
- หากเราสร้าง Class Component ไว้ในไฟล์ต่างหาก ต้องส่งออกคลาสด้วย export เช่นเดียวกับ Function Component หรือถ้าในไฟล์นั้นมีคลาสเดียว อาจส่งแบบ default ก็ได้

```
export class MyComp extends Component {  
  render() { //โอเวอร์ไรด์เมธอด render()  
    return <div>This is my component</div>  
  }  
}
```

```
//กรณีส่งออกแบบ default  
export default class MyComp extends Component {  
  render() {  
    return <div>This is my component</div>  
  }  
}
```

การสร้าง Class Component

- หากต้องการกระทำบางอย่างหลังจากที่คอมโพเนนต์แสดงผลตามที่ระบุไว้ในเมธอด `render()` ไปแล้ว ให้โอเวอร์ไรด์เมธอด `componentDidMount()` เพื่อกำหนดการกระทำที่ต้องการ ซึ่งเมธอดนี้จะถูกเรียกตามหลัง `render()` เช่น

```
export default class MyComp extends Component {  
  render() {  
    ...  
  }  
  
  componentDidMount() {  
    alert('คอมโพเนนต์แสดงผลเสร็จแล้ว')  
  }  
}
```

การสร้าง Class Component

เมื่อต้องการนำ Class Component มาใช้งานที่เพจหรือไฟล์ใด ก็ต้องอิมพอร์ตคลาสนั้นจากไฟล์ที่จัดเก็บมันเอาไว้โดยใช้วิธีใดวิธีหนึ่ง เช่น

```
//ถ้าส่งออกแบบ default  
import MyComp from './mycomponent.js'  
  
  
//ถ้าส่งออกแบบปกติ (ไม่ใช่ default)  
import {MyComp} from './mycomponent.js'
```

การสร้าง Class Component

ขั้นตอนการแสดงผล ก็ใช้แนวทางเดียวกับ Function Component นั่นคือ การระบุคลาสในรูปแบบ Single Tag เช่น `<MyComp/>` ซึ่งทดสอบการใช้งานดังนี้

ตัวอย่างการใช้งาน

`react/app1/src/class-components.js`

```
import React, {Component} from 'react'
```

//มีคลาสเดียว จึงสามารถส่งออกแบบ default ได้

```
export default class Calendar extends ..... {  
  getDate() {  
    const dayNames = ['อาทิตย์', 'จันทร์', 'อังคาร', 'พุธ', 'พฤหัสบดี', 'ศุกร์',  
                      'เสาร์']  
  
    const monthNames = ['มกราคม', 'กุมภาพันธ์', 'มีนาคม', 'เมษายน',  
                        'พฤษภาคม', 'มิถุนายน', 'กรกฎาคม', 'สิงหาคม', 'กันยายน', 'ตุลาคม',  
                        'พฤศจิกายน', 'ธันวาคม']
```

```
    const date = new Date()  
    const weekDay = dayNames[date.getDay()]  
    const day = date.getDate()  
    const month = monthNames[date.getMonth()]  
    const year = date.getFullYear() + 543  
    return `วัน${.....} ที่ ${day} ${month} ${.....}`  
  }  
  
  render() {  
    return <div>{.....}</div>  
  }  
}
```

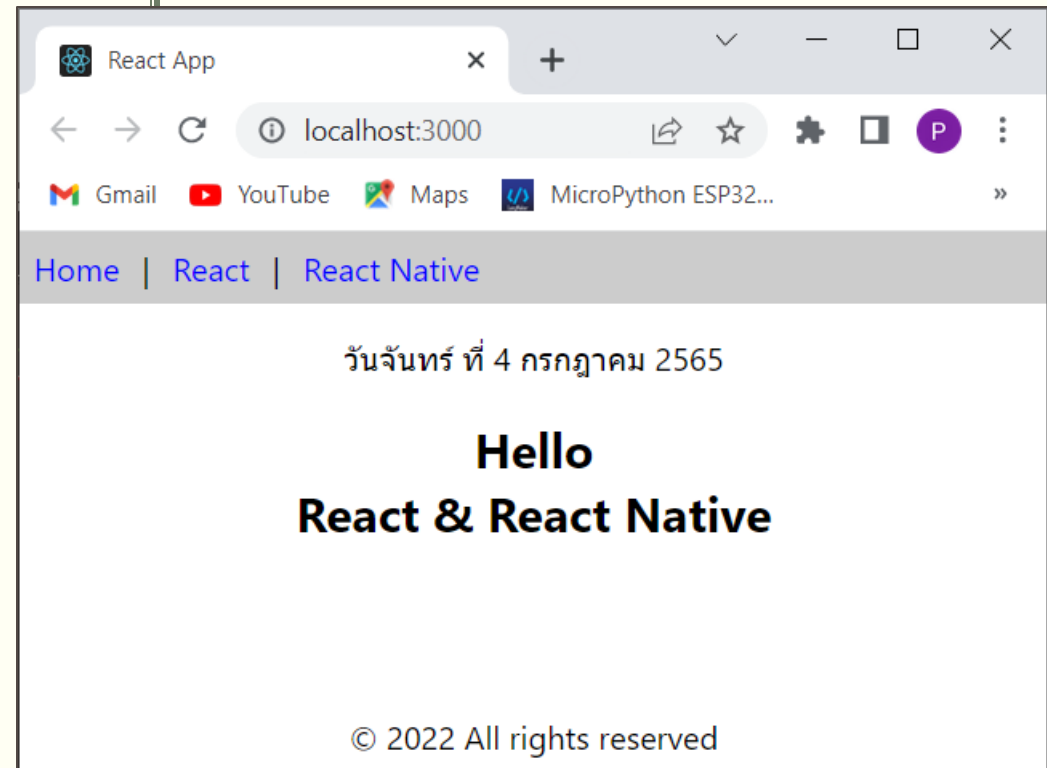
การสร้าง Class Component

react/app1/src/App.js

```
import React from 'react'
import { Header, Content, Footer } from './func-components'
import ..... from './class-components'

function App() {
  return (
    <>
      <Header/>
      <p><center>.....</center></p>
      <Content/>
      .....
    </>
  )
}

export default .....
```



จัดเก็บข้อมูลของคอมโพเนนต์ด้วย props

ข้อมูลของคลาสจะถูกจัดเก็บในรูปแบบของพร็อพเพอร์ตี้ (Property) และสำหรับ React Component ก็ใช้หลักการเดียวกัน โดยในกรณีนี้ จะเรียกมันว่า props (มาจาก properties) แต่อย่างไรก็ตาม props ของ React จะมีข้อกำหนดบางอย่างที่แตกต่างไปจากพร็อพเพอร์ตี้แบบปกติของคลาสใน JavaScript ซึ่งจะแยกอธิบายตามการใช้งานในรูปแบบ Class Component และ Function Component ดังนี้

- **การใช้ props ใน Class Component**

กรณีของ Class Component จะจัดเก็บข้อมูลไว้ในออบเจกต์ที่ชื่อ props แล้วอ้างถึงพร็อพเพอร์ตี้แต่ละค่าในรูปแบบดังนี้

`this.props.ชื่อพร็อพเพอร์ตี้`

ออบเจกต์ props สามารถเก็บข้อมูลปลีกย่อยได้หลายอย่าง ซึ่งก็คล้ายกับออบเจกต์ของ JavaScript นั่นเอง สำหรับแนวทางการรับข้อมูลเข้ามาในคลาส อาจทำได้ 2 แบบดังนี้

จัดเก็บข้อมูลของคอมโพเนนต์ด้วย props

- แบบที่ 1 รับค่าผ่านทางคอนสตรักเตอร์

โดยเราต้องกำหนดคอนสตรักเตอร์ให้กับคลาส พร้อมกับระบุพารามิเตอร์ที่ชื่อ props จากนั้นภายในคอนสตรักเตอร์ ก็ส่งต่อให้กับ Superclass ด้วยคำสั่ง super() ส่วนองค์ประกอบอื่นๆ ก็เป็นไปตามหลักการของ Class Component ที่ได้กล่าวมาแล้ว เช่น การโอเวอร์ไรด์เมธอด render() สำหรับรีอปเพอร์ตีที่อ้างอิง จะมาจากขั้นตอนที่นำคอมโพเนนต์ไปใช้งาน

```
import React, {Component} from 'react'
class MsgBox extends Component {
  constructor(props) {
    super(props)
  }
  render() {
    return <div>{this.props.text}</div>
  }
}
```

จัดเก็บข้อมูลของคอมโพเนนต์ด้วย props

- แบบที่ 2 อ้างถึง props โดยตรง

เนื่องจาก props เป็นออบเจกต์พื้นฐานที่มีอยู่ในคอมโพเนนต์ของ React อยู่แล้ว จึงสามารถอ้างถึงพร็อพเพอร์ตี้ต่างๆ ที่อยู่ใน props ได้เลย โดยไม่จำเป็นต้องรับค่าผ่านคอนสตรัคเตอร์ก็ได้

```
Class MsgBox extends Component {  
  render() {  
    return <div>{this.props.text}</div> //อ้างถึง props ได้เลย  
  }  
}
```

จัดเก็บข้อมูลของคอมโพเนนต์ด้วย props

ในขั้นตอนการนำคอมโพเนนต์ไปแสดงผล เราต้องกำหนดพรีอปรเตอร์ดีและค่าของมันในรูปแบบแอตทริบิวต์ไว้ในแท็กของคอมโพเนนต์อันนั้น เพื่อส่งให้แก่ props ของคลาส เช่น

```
<MsgBox text="Hello World"/>
```

เมื่อเรากำหนดค่าเช่นนี้ text จะกลายเป็นพรีอปรเตอร์ดีของ props โดยอัตโนมัติ และคลาสก็สามารถอ้างถึงด้วย this.props.text เพื่อนำค่าไปใช้งานได้เลย ดังโค้ดที่ผ่านมา และถ้ามีมากกว่า 1 พรีอปรเตอร์ดี ก็กำหนดเป็นแอตทริบิวต์เพิ่มเติมลงในแท็กตามเดิม เช่น

```
<MsgBox text="Hello World" color="blue" fontSize="20px"/>
```

แล้วที่คลาสก็สามารถอ้างถึงแต่ละพรีอปรเตอร์ดีด้วย this.props.text, this.props.color, this.props.fontSize อย่างนี้เป็นต้น

จัดเก็บข้อมูลของคอมโพเนนต์ด้วย **props**

ตัวอย่างการใช้งาน

react/app1/src/class-props.js

```
import React, { Component } from 'react'

export default class MsgBox extends Component {
  /*
  constructor(props) {
    super(props)
  }
  */
}
```

```
render() {
  const divStyle = {
    margin: '10px',
    padding: '5px',
    color: this.props.color,
    backgroundColor: this.props.bgColor,
    fontSize: this.props.fontSize,
    border: this.props.border
  }

  return <div style={divStyle}>{this.props.text}</div>
}
}
```

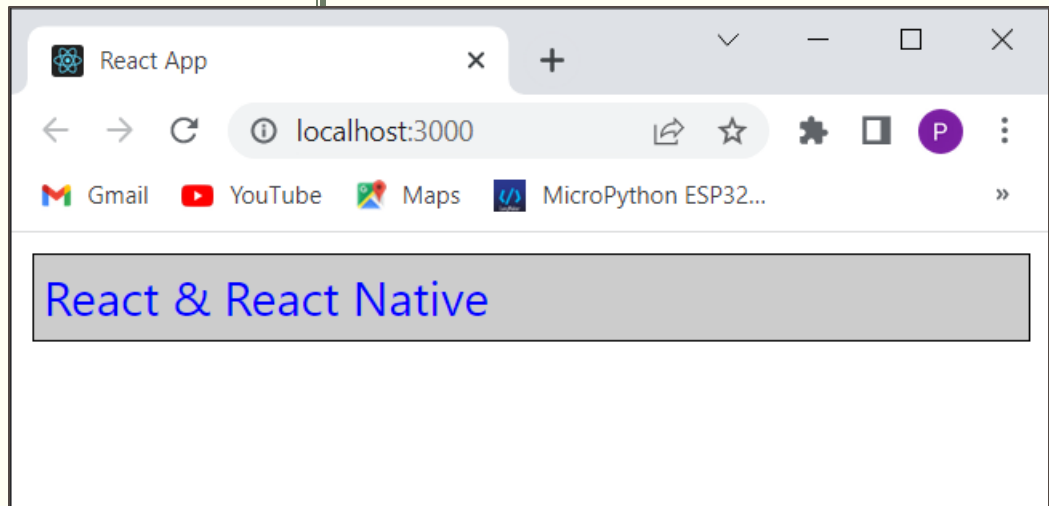
จัดเก็บข้อมูลของคอมโพเนนต์ด้วย props

react/app1/src/App.js

```
import React from 'react'
import MsgBox from './class-props'

function App() {
  return (
    <MsgBox
      text="React & React Native"
      color="blue"
      bgColor="#ccc"
      fontSize="18pt"
      border="solid 1px black"
    />
  )
}

export default App
```



การใช้ props ใน Function Component

สำหรับ Function Component หากต้องใช้ข้อมูลจาก props ก็มีวิธีแบบง่ายๆ คือ กำหนดพารามิเตอร์ props ให้แกฟังก์ชันที่สร้างขึ้น แล้วภายในฟังก์ชันก็อ้างถึงพร็อพส์ต่างๆ จากออบเจกต์ (พารามิเตอร์) ในแบบ props.ชื่อพร็อพส์ ได้เลย เช่น

```
function MsgBox(props) {  
  return <div>{props.text}</div>  
}
```

ส่วนขั้นตอนการนำไปแสดงผล ก็ทำเช่นเดียวกับกรณีของ Class Component ทั้งหมด เช่น จากโค้ดที่ผ่านมาซึ่งเราสร้างคอมโพเนนต์ MsgBox ในแบบคลาส หากเปลี่ยนมาเป็นฟังก์ชัน ก็ทำดังนี้

การใช้ props ใน Function Component

ตัวอย่างการใช้งาน

react/app1/src/func-props.js

```
import ..... from 'react'

export default function MsgBox2(.....) {
  const divStyle = {
    margin: '10px',
    padding: '5px',
    color: props.color,
    backgroundColor: .....,
    fontSize: props.fontSize,
    border: .....
  }
  return <div style={.....}>{.....}</div>
}
```

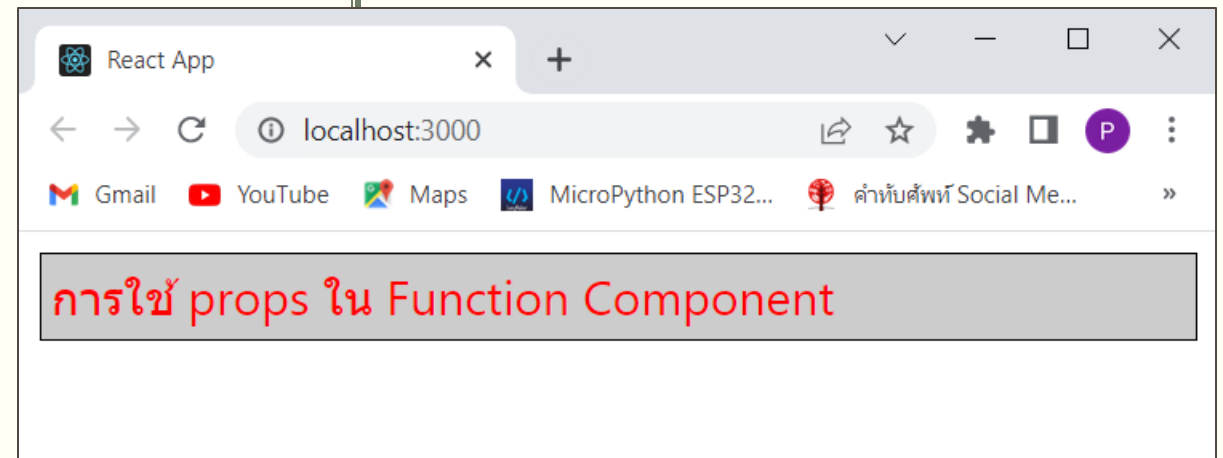
การใช้ props ใน Function Component

react/app1/src/App.js

```
import React from 'react'
import MsgBox2 from .....

function App() {
  return (
    <.....
      text= "....."
      color= "....."
      bgColor="#ccc"
      fontSize="18pt"
      border="solid 1px black"
    />
  )
}

export default App
```



สิ่งที่ควรรู้เพิ่มเติม

ข้อมูลที่จัดเก็บด้วย props จะเป็นแบบ read-only นั่นคือ สามารถอ่านค่าได้อย่างเดียว แต่ไม่สามารถเปลี่ยนแปลงค่าของมันในคลาสหรือฟังก์ชันได้ ถ้าหากจำเป็นต้องแก้ไข ให้สร้างตัวแปรมารับค่าจากมัน เช่น `let x = this.props.fontSize` แล้วแก้ไขและใช้งานผ่านตัวแปร `x` แทน